

Task -1 Web App QA & Debug Report

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Valid user credentials rejected with "Invalid" error after logout.	❖ Go to the Sign Up page and create a new account. ❖ Log out successfully. ❖ Try to log back in using the exact same username/email and password.	Expected: User logs in successfully to their dashboard. Actual: The app shows an "invalid credentials".	High	Backend session store or database dropping test accounts, or an asynchronous state issue.
2	Missing email format validation on input fields.	❖ Go to the Sign Up or Login page. ❖ Type a malformed email like omkarmail into the email field. ❖ Fill out the password and click submit.	Expected: The form blocks submission and displays a validation error ("Please enter a valid email address"). Actual: The app accepts the malformed string without any warning.	Medium	The input field is missing the HTML5 type="email" attribute, or the frontend component lacks a regular expression (check before sending the payload to the API).
3	Missing confirmation dialog before deleting an article.	❖ Log into the app and open an article you created. ❖ Click the "Delete Article" button.	Expected: A warning prompt should appear asking, "Are you sure you want to delete this article?" Actual: The article is deleted instantly upon clicking the button with zero confirmation, risking accidental data loss.	Medium	The developer omitted a confirmation state check or a modal component, and directly wired the click handler to fire the delete API request.

Task -1 Web App QA & Debug Report

4	Raw technical error message exposed on password update failure	❖ Go to the Settings page. ❖ Try updating your password with a short or invalid format, or trigger a validation rule. ❖ Submit the form.	Expected: A clean, user-friendly error message (e.g., "Password must be at least 8 characters long"). Actual: A raw API error object string is displayed directly (body password must be a string between 8 and 128 chars).	Medium	The frontend component is directly rendering the raw JSON response error string or validation middleware message from the backend API instead of mapping it to a friendly UI notification.
5	Insecure password policy allowing single-character passwords	❖ Go to Settings or Register page. ❖ Type a single character (e.g., a) into the password field. ❖ Submit the form.	Expected: The app should reject the password and show an error stating passwords must be at least 8 characters long. Actual: The app accepts the single-character password and saves it successfully.	High	Missing frontend length validation checks and insecure backend database schema constraints that do not enforce a minimum password length.

Conduit

Home [New Article](#) [Settings](#)  Omkar

Your Settings

• body password must be a string between 8 and 128 chars

URL of profile picture

Om

Short bio about you

omkarparademail

....

Update Settings

URL: <https://demo.realworld.show>

Task -1 Web App QA & Debug Report

Valid user credentials rejected with "Invalid" error after logout

The issue where valid user credentials are rejected with an 'invalid' error immediately after logging out stems from an inconsistency between client-side session destruction and backend state synchronization. When a user logs out, the frontend typically clears local storage or session tokens, but the backend session store or authentication database may prematurely invalidate user records if the application is running on a volatile shared test environment. Alternatively, asynchronous token blacklisting or caching layers might retain stale authentication states, causing subsequent login attempts to be erroneously rejected by the API middleware. To fix this behaviour, a developer must ensure that the backend authentication service cleanly handles persistent user records without wiping valid user accounts during session termination. Additionally, implementing robust token lifecycle management and proper error handling on the login endpoint will ensure that genuine user accounts can authenticate reliably without unexpected database desynchronization.